

IMY 220

Assignment 2

React Basics

Due: Wednesday, 19 August 2026 at 13:00

The submission instructions are available on ClickUP. Any deviation from these instructions will result in a 10% deduction from your mark.

Instructions

This is an individual, take-home assignment.

This assignment focuses on:

- functional components;
- JSX;
- props;
- component state;
- event handling;
- forms and user input;
- rendering arrays using `map()`;
- updating arrays stored in state.

React components are reusable functions that receive data through props and return JSX. Components may also be nested to create larger interfaces.

Scenario

You have joined the development team responsible for the PhotoShare social media platform.

Another developer has already created the basic project structure and supplied an initial collection of posts.

Your task is to complete a React application that allows a user to view, search, sort and manage PhotoShare posts.

Supplied Files

Download the supplied files from ClickUP:

index.html

This file contains the required React, ReactDOM and Babel setup.

Do not modify the supplied React imports or the `<div id="root"></div>` element.

Open the application using a local development server, such as the VS Code Live Server extension. Do not open `index.html` directly from the file system.

You may update the styling or add additional styling if required.

App.js

This file contains:

- the initial array of posts;
- the basic App component;
- the React rendering code.

Add all your React components and functionality inside this file.

Replace `uXXXXXXX` with your student number.

styles.css

This file contains the supplied styling for the application.

You may modify and extend the styling as required.

Ensure that your student number is included as a comment in `index.html`, `App.js` and `styles.css`.

Supplied Post Data

Each post follows this structure:

```
{
  id: 1,
  username: "@sarah",
  caption: "Studying React today!",
  status: "Published"
}
```

Each post contains:

- a unique `id`;
- a `username`;

- a caption;
- a status.

The following statuses are used:

- Draft
- Published
- Archived

Do not add, remove or update items in the supplied posts array. Use it only as the initial value for state in the App component.

State must be updated through its state setter so that React can update the displayed interface.

Required Component Structure

Your application must include the following functional components:

```
App
├── SearchBar
├── SortControl
├── AddPostForm
├── PostList
│   └── PostCard
```

You may create additional components if required.

All components must be implemented as functional components, not class components.

Do not use JavaScript imports or exports. The React library has already been included for you in `index.html`.

Section 1: Display the Posts

Store the supplied array of posts in the state of the App component.

Create a component called `PostCard`.

The `PostCard` component must receive the following information through props:

- `id`;
- `username`;

- `caption`;
- `status`;
- the functionality required to delete a post;
- the functionality required to change a post's status.

The component must display:

- the username inside an `<h2>` element;
- the caption inside a `<p>` element;
- the current post status;
- a button used to delete the post;
- a button used to change the post status.

Create a component called `PostList`.

The `PostList` component must receive the posts through props.

Use `map()` inside `PostList` to generate one reusable `PostCard` component for every post.

Each generated `PostCard` must have an appropriate, unique key.

React uses keys to identify items that have been added, changed or removed from a rendered array.

When there are no posts to display, the `PostList` component must display an appropriate message such as:

No posts found.

Section 2: Search for Posts

Create a component called `SearchBar`.

Add a controlled text input that allows the user to search for posts.

The search must check both:

- the post username;
- the post caption.

Searching must be case-insensitive.

For example:

- searching for **sarah** should display posts created by @sarah;
- searching for **react** should display posts containing “React” in their caption.

The displayed posts must update immediately while the user types.

When the search input is empty, all posts must be displayed.

When no posts match the search, display an appropriate message such as:

No posts found.

React state may be used to store and update values entered into input fields.

Section 3: Sort the Posts

Create a component called `SortControl`.

The component must contain a sorting control that allows the user to choose between:

- Username A–Z;
- Caption A–Z.

The selected sorting option must be stored in state.

The displayed posts must update when the selected sorting option changes.

The sorting must be applied to the posts currently matching the search.

For example, when a user searches for `react` and selects **Username A–Z**, only matching posts should be displayed, sorted alphabetically by username.

Do not directly modify the original posts array when sorting.

The selected sorting option should remain active while searching.

Section 4: Add a Post

Create a component called `AddPostForm`.

The form must contain the following inputs:

- Username;
- Caption.

Both inputs must be controlled using React state.

When the form is submitted:

1. Prevent the page from refreshing.
2. Validate that both the username and caption contain a value.
3. If either input is empty, the post must not be added.
4. Create a new post object.
5. Generate a unique `id` for the post.
6. Set the post status to `Draft`.
7. Add the new post to the posts stored in state.
8. Clear both form inputs.
9. The search and sorting functionality should continue to work correctly after a new post has been added.

The new post should use the following structure:

```
{
  id: generatedId,
  username: enteredUsername,
  caption: enteredCaption,
  status: "Draft"
}
```

The new post must appear in the feed immediately after it has been added.

React event handlers receive functions, and the default submission behaviour of a form can be cancelled using `event.preventDefault()`.

Section 5: Delete a Post

Add a **Delete Post** button to every `PostCard`.

When the button is clicked, only the selected post must be removed from the posts stored in state.

The updated list must appear immediately.

The delete functionality should be implemented in the `App` component and passed to the appropriate child components using props.

Section 6: Change the Post Status

Add a **Change Status** button to every PostCard.

When clicked, the post status must cycle in the following order:

Draft → Published → Archived → Draft

Examples:

- A post with the status Draft becomes Published.
- A post with the status Published becomes Archived.
- A post with the status Archived becomes Draft.

Only the selected post should be updated.

The status update functionality should be implemented in the App component and passed to the appropriate child components using props.

Props allow a parent component to pass both data and functionality to child components.

Section 7: Application Structure and Styling

Your application must:

- use all six required components;
- use props to pass data and functionality between components;
- use state for values that change;
- use controlled inputs for the search and add-post form;
- use `map()` to generate the post cards;
- use unique keys for rendered posts;
- avoid directly modifying arrays stored in state;
- display an appropriate message when there are no posts to display;
- remain fully functional after posts are added, deleted or updated.

Apply basic styling to make the application clear and user-friendly.

The styling does not need to match a supplied example exactly, but the following parts should be visually distinguishable:

- page heading;
- search and sorting controls;
- add-post form;
- individual post cards;
- status labels;

- buttons;
- empty-results message.

Use reusable components where appropriate. Avoid duplicating JSX.

Expected Application Behaviour

When the page first loads:

- all supplied posts are displayed;
- posts are generated using reusable PostCard components;
- each post displays its username, caption and status.

The user must then be able to:

- search by username or caption;
- sort by username or caption;
- add a new post;
- delete a post;
- change a post's status.

All changes must update the interface without refreshing the page.

Submission Files

Submit only the following files:

index.html
App.js
styles.css

Submission Instructions

ZIP all submission files using a tool such as 7-Zip or WinRAR.

Name the ZIP file:

a2-{student_number}.zip

Replace {student_number} with your student number.

Submit the ZIP file before the deadline to avoid a late submission.